OPIO

Monitoring & Observability Audit

Guillaume · May 7, 2026

Follow-up to the May 5 discussion with Olivier

Contents

1. Context & Scope

Olivier raised two questions on May 5:

1. **Visibility** — "Where do I find the top 10 slowest requests?"
2. **Reactivity** — "How do we build a system that detects and acts fast?"

This audit answers both by examining the full observability stack: Sentry, Application Insights, OpenTelemetry, Langfuse, and the code that wires them together. Scope: all four services (API, Dagster, Temporal, Frontend), Helm infrastructure (8 client helmfiles, 3 deployment templates), Terraform, and CI.

Methodology. Sentry API queries (projects, issues, alert rules, environments, SDK versions, event volumes), systematic codebase review (every `sentry_sdk.init()`, every `configure_azure_monitor()`, every middleware and error handler), Helm chart analysis, and CI workflow configuration.

2. Executive Summary

The monitoring system exists but has never functioned as a monitoring system.

> **348 issues** across 3 Sentry projects since February 2026. **Zero** have ever been triaged — not one resolved, ignored, or assigned.

348	98.7%	0
open issues — zero triaged	of Sentry events are Dagster noise	performance traces ever recorded

- **Dagster consumes 98.7%** of all error events (234,757 / 237,922 in 30 days). A single broken SharePoint connection generates 143,686 events. Quota exceeded — real errors from other services silently dropped.
- **Temporal worker** — 80% of real workload (skills, master planner, HITL, quality gates) — sends nothing to Sentry. DSN injected by Helm; code never reads it.
- **Zero performance data** recorded. `firstTransactionEvent: False` on all 3 projects. The "top 10 slowest requests" question is unanswerable.
- **PII flows freely** to Sentry: `send_default_pii=True`, `dataScrubber: False`, JWT tokens logged in error extras. SOC2/ISO27001 compliance gap.

3. Current Architecture

WHAT EXISTS

Service	Sentry	AppInsights	Langfuse	OTel
API (FastAPI)	✓ errors only	✓ + instrumentors	—	Via AI
Dagster	✓ errors only	dead code	—	dead code
Temporal worker	× not init'd	not init'd	✓ optional	1 counter
Frontend (React)	✓ errors only	✓ analytics	—	—

SENTRY PROJECTS

Project	SDK	Events/30d	Perf	Issues	Triaged
`opio-api`	python.fastapi v2.52	2,915	×	83	0
`opio-dagster`	python v2.52	**234,757**	×	216	0
`opio-front`	react v10.43	250	×	49	0
Temporal worker	**none**	0	×	—	—

ENVIRONMENTS

10 tenant environments visible: `hackaton-prod`, `demo-prod`, `squareness-prod`, `nexia-prod`, `bdo-prod`, `ardian-prod`, `aurys-prod`, `saas-prod`, `saas-bdo-prod`, `production`. All `*-prod` — every staging has `sentryEnv: ""` → Sentry disabled.

ALERT RULES (IDENTICAL ACROSS ALL 3 PROJECTS)

Rule	Trigger	Action	Cooldown
Slack	New issue created	`#sentry-bugs`	**24 hours**
Email	High priority	IssueOwners / ActiveMembers	30 min

No regression detection. No spike detection. No per-environment filtering.

4. Findings

CRITICAL **— F1 · Dagster is drowning Sentry (active incident)**

`scan_dataroom_sensor` runs every 5 minutes, iterates all active SharePoint connections, no circuit breaker. Broken connection → retry indefinitely → events flood.

Events	Env	Issue
143,686	nexia-prod	SharePoint auth failed — scan_dataroom_sensor
10,552	—	SharePoint sensor: Error checking changes
6,443	—	synchronize_dataroom_scan_job failure
5,864	—	**Sentry ingestion suspended** (quota 403)
5,148	—	scan_dataroom RUN_FAILURE (active May 7)
5,086	—	**Redis ping failed** (active May 7)
4,492	—	synchronize_dataroom_job failure
4,178	—	index_document_job failure (active)

Impact: Quota exhausted → ingestion suspended → errors from API, Frontend silently dropped. The monitoring system is monitoring itself failing.

CRITICAL **— F2 · Temporal worker is invisible**

`temporal/worker.py` never calls `sentry_sdk.init()`. Helm injects `SENTRY_DSN` + `SENTRY_ENV` — code never reads them.

All skill executions (LLM calls, sandbox runs, result extraction), master planner decisions, HITL interactions, and quality gates produce zero events.

Additional concern: Temporal DSN is `sentryDagsterDsn` — same as Dagster. Needs its own `opio-temporal` project.

CRITICAL **— F3 · Zero performance monitoring**

`firstTransactionEvent: False` on all 3 projects. No trace ever recorded.

Missing in each SDK init:

- API (`main.py`): no `traces_sample_rate`
- Dagster (`opioflows/__init__.py`): no `traces_sample_rate`
- Frontend (`index.tsx`): no `BrowserTracing` integration

Impact: "Where are the top 10 slowest requests?" → unanswerable.

HIGH **— F4 · PII flowing to Sentry (compliance)**

Setting	Value	Risk
`send_default_pii=True`	API + Dagster	IPs, emails, cookies to Sentry
`dataScrubber` (org)	`False`	No server-side scrubbing
`scrubIPAddresses` (org)	`False`	Client IPs stored
`enhancedPrivacy` (org)	`False`	Enhanced privacy OFF
Auth error logging	JWT in extras	Token visible in Sentry

SOC2/ISO27001 certification in progress — these settings constitute an audit finding.

HIGH **— F5 · Auth double-reporting bug**

`auth/dependencies.py:get_current_user()` — broad `except Exception` catches its own `HTTPException` re-raise.

Every JWT failure generates **two** Sentry events:
1. "Invalid token: JWT validation failed" — 7,763 events
2. "Error getting current user" — 12,900 events

Combined: 20,600 events from one function. `extra={"token": str(token)}` also logs full JWT token — PII leak.

HIGH **— F6 · Alerting produces no behavior change**

24-hour cooldown on Slack rule: new issue fires once, goes to `#sentry-bugs`, nobody looks. After 24h, issue is no longer "new" — rule never fires again.

Result: A new issue gets exactly one Slack notification, ever. The high-priority email targets "IssueOwners" — no issues have owners, falls back to all 5 members (diffusion of responsibility).

MEDIUM **— F7 · All staging blind**

Every helmfile: `sentryEnv: ""` → guard `if sentry_dsn and sentry_env:` fails → Sentry disabled on all staging environments.

MEDIUM **— F8 · Dead observability code**

Module	Purpose	Status
`shared/error_tracking/`	AppInsights error capture	Never imported
`shared/analytics/`	AppInsights event tracking	Never imported
`OrchestratorTelemetryService`	OTel metrics for Dagster	6 `track_*()` never called

`SentryContextMiddleware` opens a separate DB session per request for user lookup — duplicates `get_current_user()`.

MEDIUM **— F9 · AppInsights is self-destructing**

`configure_azure_monitor()` generates Sentry errors since February: `HTTPSConnectionPool timeout` — 381 events, still active May 6. The monitoring tool is generating errors in the monitoring system.

5. Risk Assessment

Category	Description	Level
Operational	Dagster quota flood → real errors dropped. Pipeline failure = zero Sentry events.	HIGH
Compliance	SOC2 CC7.2 / ISO 27001 A.12.4: 348 unresolved issues, PII in logs, no triage process.	HIGH
Business	Gilbert agentic workflow depends on fast error detection. Temporal blind spot + 24h cooldown breaks the feedback loop.	MEDIUM

6. Recommendations

PHASE 0 — STOP THE BLEEDING (TODAY, 2 HOURS)

#	Action	Effort	Impact
0.1	Resolve/mute 5 noisiest Dagster SharePoint issues	10 min	Reclaim 200K/mo quota
0.2	Fix auth double-reporting: exclude `HTTPException` from catch	15 min	Eliminate 20K phantom events
0.3	Remove JWT token from error log extras	5 min	Stop PII leak
0.4	Enable `dataScrubber`, `scrubIPAddresses` in Sentry org	5 min	Compliance quick fix

> **Result:** 95% of noise eliminated. Sentry becomes usable.

PHASE 1 — ESTABLISH VISIBILITY (THIS WEEK, 1 DAY)

#	Action	Effort
1.1	Add `sentry_sdk.init()` to Temporal worker	30 min
1.2	Create `opio-temporal` Sentry project, separate DSN	15 min
1.3	Enable `traces_sample_rate=0.1` on API, Dagster, Temporal	15 min
1.4	Add `BrowserTracing + Replay` to frontend	30 min
1.5	Set `sentryEnv` on staging (demo + bdo)	10 min
1.6	Add `release` tag to API + Dagster SDK init	15 min
1.7	Set `send_default_pii=False` on API + Dagster	5 min

> **Result:** All four services visible. Performance data flows. Staging covered. Compliance-safe.

PHASE 2 — BUILD THE FEEDBACK LOOP (NEXT WEEK, 2 DAYS)

#	Action	Effort
2.1	Replace alert rules: `#sentry-critical` (no cooldown), regression detection	1h
2.2	Triage 348 existing issues: resolve ancient, group duplicates, assign owners	2h
2.3	Create Sentry dashboards: overview + per-tenant health	1h
2.4	Circuit breaker for SharePoint sensor (disable after N failures)	2h
2.5	Refactor SentryContextMiddleware → set user in `get_current_user()`	1h
2.6	Delete dead code: `error_tracking/`, `analytics/`, `track_*()`	30 min

PHASE 3 — ORGANIZATIONAL PRACTICES (WEEKS 2–4)

The technical fixes from Phase 0–2 are necessary but insufficient. Without process, the Sentry dashboard becomes another ignored tab. This section proposes the minimal viable practices to turn monitoring data into team behavior.

Triage ritual

Institute a **weekly 30-minute bug review**. One person shares their screen on Sentry, the team walks through new/unresolved issues. Goal: every issue leaves the meeting with a status (resolved, ignored with reason, assigned to someone, or escalated). This is the single highest-leverage practice — it's what makes monitoring exist as a function rather than an artifact.

Suggested format:

- **Monday morning, 30 min**, before sprint work begins.
- Walk the Sentry dashboard: new issues since last week → assign or resolve.
- Quick check: per-tenant health (any tenant degrading?).
- Output: Linear tickets for anything that needs code changes.

Ownership model

Assign **service owners** in Sentry:

Project	Owner	Scope
`opio-api`	TBD	Auth, routers, middleware, DB pool
`opio-dagster`	TBD	Sensors, jobs, SharePoint, document pipeline
`opio-temporal`	TBD	Skills, master planner, HITL, agent tasks
`opio-front`	TBD	UI errors, network failures, PDF rendering

Owners don't fix every bug — they **triage**. Their job is to decide: is this worth fixing now, later, or never? When an alert fires, the owner is the first responder.

Alerting tiers

Replace the current flat "everything → `#sentry-bugs`" with a tiered system:

Tier	Channel	Trigger	Response
P0	`#sentry-critical`	New issue affecting >10 users in <1h, or ingestion suspended	Immediate — owner investigates within 15 min
P1	`#sentry-alerts`	New issue in any prod env, or resolved issue regressing	Same-day — owner triages within 4h
P2	Weekly triage	All other issues	Weekly bug review decides priority

Key change: **P0 has no cooldown** and pings the service owner directly. The current 24h cooldown is eliminated for critical alerts.

Sentry → Linear integration

Connect Sentry to Linear so that triaged issues become trackable work items. Sentry supports this natively: the "Create Issue" button in Sentry UI can push directly to a Linear project. This bridges the gap between "we know about the bug" and "someone is assigned to fix it."

Suggested workflow:

1. Sentry alert fires → owner triages in weekly review (or immediately for P0).
2. If fix needed → "Create Linear Issue" from Sentry UI. Sentry auto-links the issue.
3. Developer fixes → PR merged → Sentry issue auto-resolves on next deploy (via `release` tag).
4. If issue regresses → Sentry re-opens and fires a regression alert.

Monitoring as definition of done

Add to the PR checklist (`.github/PULL_REQUEST_TEMPLATE.md`):

- [ ] Errors are handled and tracked (no bare `except: pass`)
- [ ] New services have `sentry_sdk.init()` configured
- [ ] No PII in error logs or Sentry extras

For Gilbert/agentic workflow specifically: when an agent creates a PR, the CI pipeline should deploy to staging (with Sentry enabled after Phase 1.5) and verify no new Sentry issues appear. This closes the loop: agent writes code → CI checks for errors → monitoring catches what tests miss.

PHASE 4 — BUSINESS METRICS & DEEP INSTRUMENTATION (MONTH 2)

Beyond error tracking, monitoring should answer **business** questions:

Pipeline throughput

Metric	Why it matters
Documents processed / hour / tenant	Olivier's "first value too slow" concern — track it, set baselines, alert on degradation
Skill success rate (% of agent tasks completing without error)	Agentic reliability — if skills fail silently, users wait forever
Time-to-first-value (upload → first usable block)	The metric users feel. Target: <15 min for simple projects
LLM cost per operation (model × tokens × price)	Budget control as usage scales across tenants
Queue depth (Temporal pending tasks)	Early warning for capacity issues — if queue grows, processing lags

Implementation: custom Sentry spans (`sentry_sdk.start_span()`) around key operations. The data feeds both Sentry's Performance dashboard and custom Sentry Discover queries.

Per-tenant health dashboard

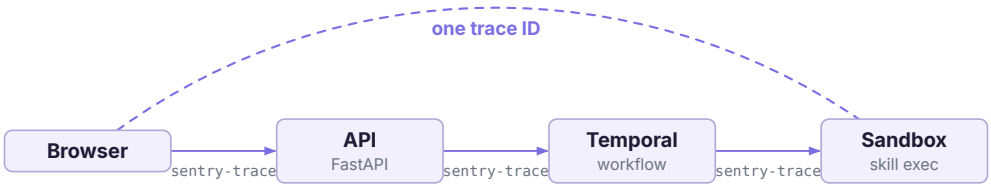
Create a Sentry dashboard filterable by `environment` tag (= tenant). Each tenant gets a health card showing:

- Error rate trend (last 7d)
- Top unresolved issue
- Pipeline throughput (if Phase 4 metrics are instrumented)
- Last deployment version

This answers the question: **"Is tenant X healthy?"** — before the client reports a problem. For the SOC2 story, it also demonstrates proactive monitoring.

End-to-end tracing

Propagate trace context across service boundaries so a single trace ID follows a request from browser to sandbox:



This lets you answer: "User clicked 'Generate GL' — what happened across all services, and where did it slow down?" Requires `sentry-trace` header propagation (FastAPI → Temporal client → activity context).

PHASE 5 — AI-ASSISTED OPERATIONS (QUARTER 2)

Automated triage

Leverage the existing Temporal + Claude infrastructure:

```
Sentry webhook → Temporal workflow → Claude
  ├─ Reads: stacktrace, tags, affected envs, event count
  ├─ Pulls: relevant source code from repo
  ├─ Produces: diagnosis + suggested fix + severity
  └─ Posts: structured summary to Slack + creates Linear ticket
```

Start with a **weekly automated summary** (Monday morning, before the triage ritual): Claude reads all new issues from the past week, groups them by root cause, and posts a digest to `#sentry-triage` . The team reviews the AI summary instead of raw Sentry. Evolve to real-time per-issue triage as confidence grows.

Proactive client outreach

When a tenant's error rate spikes above its 7-day baseline, automatically notify the account owner (not just the dev team). Template: *"We detected increased errors in your environment and are investigating. No action needed on your side."* This turns monitoring from an internal tool into a client trust signal — especially relevant for the SOC2 story and the "sell the tool to PE funds" vision.

Anomaly detection

Sentry's metric alerts support anomaly detection (deviation from historical baseline). Configure for:

- Error rate spikes per tenant (relative to that tenant's normal)
- Pipeline throughput drops (fewer docs processed than usual)
- Response time P95 increases (after traces are enabled)

This catches problems that fixed thresholds miss — e.g., a tenant that normally processes 100 docs/day suddenly drops to 10.

7. Stack Decision

Tool	Role	Action
Sentry	Errors, perf, alerting, replay, dashboards	**Primary** — fix config, enable traces
AppInsights	Azure infra metrics (AKS, PG, Redis)	**Secondary** — remove wrappers, keep auto
Langfuse	LLM call tracing	**Keep** — well-scoped, env-gated
OTel	Metrics/traces transport layer	**Keep** — foundation for future

8. Appendix

SENTRY EVENT VOLUME BY TENANT (DAGSTER, CUMULATIVE SINCE FEB 2026)

Tenant	Events	Share
nexia-prod	167,148	51.7%
aurys-prod	94,610	29.3%
demo-prod	38,220	11.8%
bdo-prod	12,293	3.8%
hackaton-prod	6,983	2.2%
squareness-prod	3,376	1.0%
ardian-prod	761	0.2%

Total: 323,391 events. Source: Sentry tag values (cumulative since project creation, February 2026).

SENTRY ORG SETTINGS

Setting	Current	Recommended
`dataScrubber`	`False`	`True`
`dataScrubberDefaults`	`False`	`True`
`scrubIPAddresses`	`False`	`True`
`enhancedPrivacy`	`False`	`True`
`require2FA`	`False`	`True`

MEMBERS

5 members: Olivier (owner), Antoine, Maria, Hilaire (managers), Guillaume (member).